

Sorting Algorithms & Performance Timing

In this lab you will implement five classic sorting algorithms — Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort — and measure how long each one takes to run using C++'s `<chrono>` library. For every algorithm you will sort arrays of increasing size, record the timing across 5 trials, and compare the results to what Big-O complexity predicts.

Setup — A Reusable Timer Utility

Before writing any sorting algorithm, add the following utility code to the top of your program. You will call `startTimer()` and `stopTimer()` (or wrap your sort call as shown) around each sorting function to measure its execution time in milliseconds.

```
#include <iostream>
#include <chrono>
#include <array>
using namespace std;
using namespace std::chrono;

int main() {
    array<int> arr = {5, 3, 8, 1, 2};
    //start timer
    auto start = chrono::high_resolution_clock::now();
    bubbleSort(arr);
    // STOP TIMER
    auto end = chrono::high_resolution_clock::now();
    // CALCULATE TIME
    auto duration = chrono::duration_cast<chrono::microseconds>(end - start);
    cout << "Sorting time: " << duration.count() << " microseconds\n";
}
```

Note: `timeSort()` takes the array by value, so each call to `sortFn` receives its own fresh, unsorted copy — you don't need to re-generate the array between algorithms if you want to compare them on identical data.

Notice: each function you must separately into header file (name.hpp) and we must test it in to main.cpp

Exercise 1 — Bubble Sort

TASK

- Repeatedly step through the array, comparing each pair of adjacent elements.
- Swap them if they are in the wrong order.
- Repeat the pass until no swaps are needed (or the array is fully traversed $n-1$ times).
- Time it using `timeSort()` for each array size and fill in the table below.

STARTER CODE

```
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
```

```

        int temp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
    }
}
}

```

TIMING TABLE — run 5 trials, record the sort time using the timer utility above

Trial	Array Size (n)	Time (ms)	Notes
1	2000	9.0	
2	4000	37.0	
3	8000	166.4	
4	16000	720.6	
5	32000	2944.6	

Observation: as n grows, how does the time change? Does this match the algorithm's expected Big-O complexity?

Exercise 2 — Selection Sort

TASK

- For each position i from 0 to n-1, find the index of the minimum element in arr[i..n-1].
- Swap that minimum element into position i.
- Repeat until the array is fully sorted.
- Time it using timeSort() for each array size and fill in the table below.

STARTER CODE

```

void selectionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }
        if (minIndex != i) {
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
}

```

TIMING TABLE — run 5 trials, record the sort time using the timer utility above

Trial	Array Size (n)	Time (ms)	Notes
1	2000	4.0	
2	4000	12.9	

Trial	Array Size (n)	Time (ms)	Notes
3	8000	52.9	
4	16000	215.2	
5	32000	875.9	

Observation: as n grows, how does the time change? Does this match the algorithm's expected Big-O complexity?

Exercise 3 — Insertion Sort

TASK

- Starting from the second element, take each element (the 'key') and insert it into its correct position among the already-sorted elements to its left.
- Shift larger elements one position to the right to make room.
- Time it using `timeSort()` for each array size and fill in the table below.

STARTER CODE

```
void insertionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

TIMING TABLE — run 5 trials, record the sort time using the timer utility above

Trial	Array Size (n)	Time (ms)	Notes
1	2000	3.0	
2	4000	9.7	
3	8000	37.0	
4	16000	149.0	
5	32000	608.1	

Observation: as n grows, how does the time change? Does this match the algorithm's expected Big-O complexity?

Bonus Challenge — Compare All Five

TASK

- Write a single program that generates one random array per size and runs all 5 sorts on identical copies of it (recall `timeSort()` takes the vector by value, so this is safe).
- Print a results table to the console with columns: Array Size | Bubble | Selection | Insertion | Merge | Quick (all times in ms).
- Fill in the summary table below using your program's output for size $n = 10,000$.

SUMMARY TABLE — $n = 10,000$

Algorithm	Trial 1 (ms)	Trial 2 (ms)	Trial 3 (ms)	Avg (ms)

Discussion: Which algorithm was fastest? Which was slowest? Do the results match the expected time complexities — $O(n^2)$ for Bubble/Selection/Insertion vs. $O(n \log n)$ for Merge/Quick? Under what conditions might Insertion Sort actually outperform Quick Sort?